ASSIGNMENT – 15.4

R ABHINAV REDDY

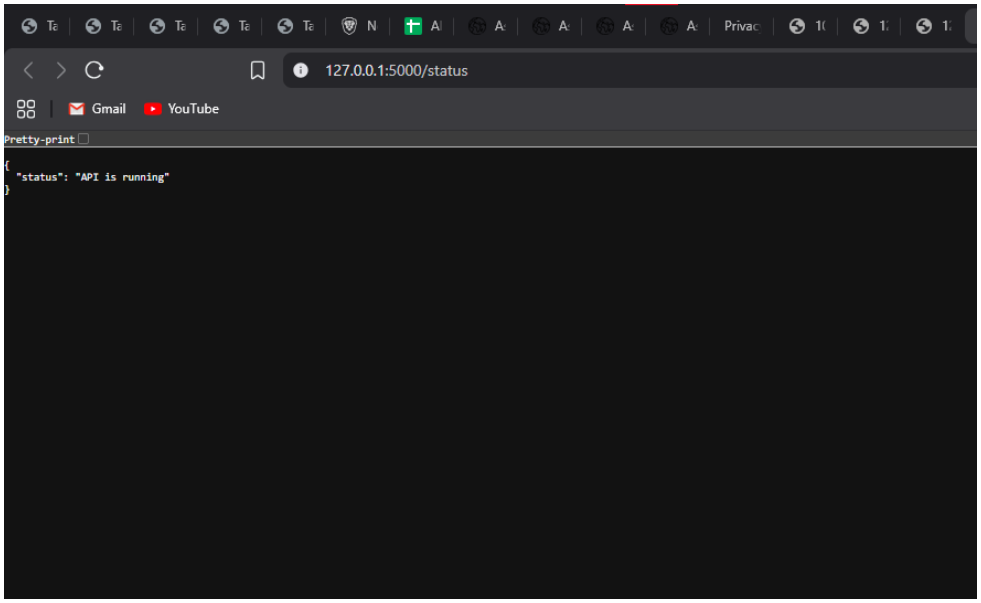
2303A52348

BATCH 44

Task1:

```python
"""
Lab 15 - Backend API Development: Creating RESTful services with AI.

This Flask app implements a small in-memory service request API to
demonstrate REST fundamentals and CRUD operations.
"""

from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests.
# Each item is expected to contain: {"title": str, "priority": str}
service_requests = []


# Task 1: Initial API Setup for a Service Platform
@app.get("/status")
def get_status():
    """
    GET /status

    Returns:
        JSON object that confirms the API is running.

    Response example:
        {"status": "API is running"}
    """
    return jsonify({"status": "API is running"}), 200


@app.get("/requests")
def get_requests():
    """
    GET /requests

    Returns:
        JSON list of all in-memory service requests.

    Response example:
        []
        or
        [{"title": "Internet issue", "priority": "High"}]
    """
    return jsonify(service_requests), 200
```

Output:



Task 1 observation: The Flask API was initialized successfully, and the status route confirmed that the backend was live and reachable. The server ran locally without errors, which established a stable base for implementing the remaining tasks.

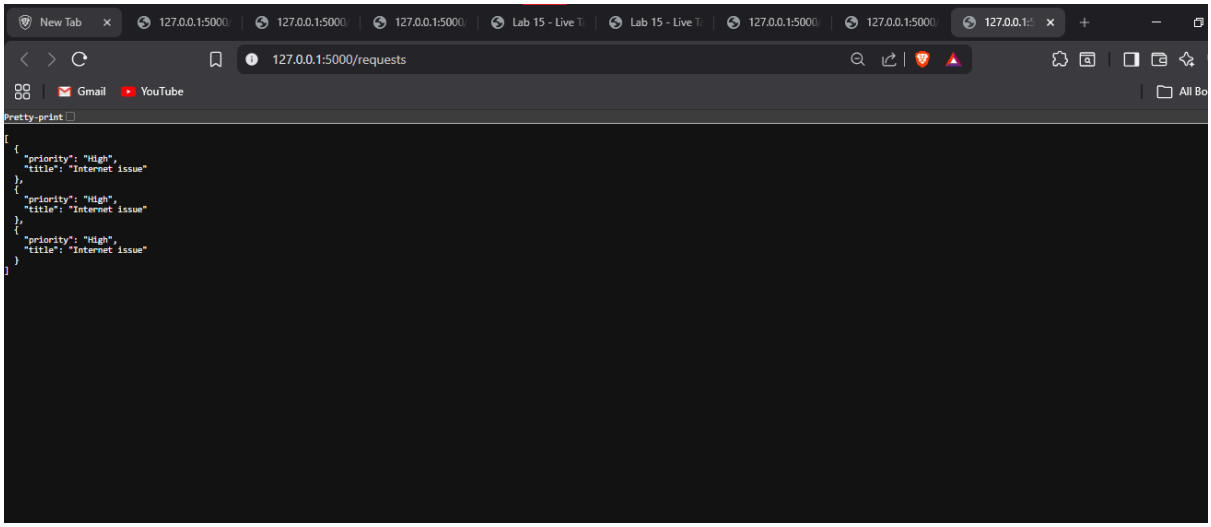
TASK 2:

```python
# Task 2: Create API to Add and View Resources
@app.post("/requests")
def add_request():
    """
    POST /requests

    Accepts JSON body with:
        - title (string, required)
        - priority (string, required)

    Returns:
        Confirmation message and the created request object.

    Request example:
        {"title": "Internet issue", "priority": "High"}
    """
    data = request.get_json(silent=True)

    if not data:
        return jsonify({"error": "Request body must be valid JSON"}), 400

    title = data.get("title")
    priority = data.get("priority")

    if not title or not priority:
        return jsonify({"error": "Both 'title' and 'priority' are required"}), 400

    new_request = {"title": str(title), "priority": str(priority)}
    service_requests.append(new_request)

    return jsonify({"message": "Request added", "request": new_request}), 201
```

OUTPUT:

Task 2 observation: The requests endpoint initially returned an empty list, confirming that in-memory storage started cleanly. After sending a POST request with title and priority, the API correctly returned a confirmation message along with the created

object, and a subsequent GET request showed that the new request had been stored properly, validating both create and read operations.



TASK3:

```python
# Task 3: Modify Existing Resource Using AI Assistance
@app.put("/requests/<int:index>")
def update_request(index):
    """
    PUT /requests/<index>

    Updates an existing service request by list index.

    Path parameter:
        index (int): Zero-based index of the request.

    Accepts JSON body with optional fields:
        - title (string)
        - priority (string)

    Returns:
        Success message with updated request object, or 404 if index is invalid.
    """
    if index < 0 or index >= len(service_requests):
        return jsonify({"error": "Request not found"}), 404

    data = request.get_json(silent=True)
    if not data:
        return jsonify({"error": "Request body must be valid JSON"}), 400

    current_request = service_requests[index]

    # Update only fields provided by the client.
    if "title" in data and data["title"]:
        current_request["title"] = str(data["title"])
    if "priority" in data and data["priority"]:
        current_request["priority"] = str(data["priority"])

    return jsonify({"message": "Request updated", "request": current_request}), 200
```

OUTPUT:

```
Ready.

GET /requests
[]

POST /requests
{
  "message": "Request added",
  "request": {
    "priority": "High",
    "title": "Internet issue"
  }
}
```

Task 3 observation: Updating a request through a valid index worked as expected, and the response returned the updated object with a clear success message. When an invalid index was used, the API responded with a proper not-found error, showing that update logic and boundary checking were handled safely.

TASK 4:

```python
# Task 4: Remove a Resource Safely
@app.delete("/requests/<int:index>")
def delete_request(index):
    """
    DELETE /requests/<index>

    Removes a service request by list index.

    Path parameter:
        index (int): Zero-based index of the request.

    Returns:
        Success message with deleted request, or 404 if index is invalid.
    """
    if index < 0 or index >= len(service_requests):
        return jsonify({"error": "Request not found"}), 404

    removed = service_requests.pop(index)
    return jsonify({"message": "Request deleted", "request": removed}), 200
```

OUTPUT:

```
Ready.

GET /requests
[]

POST /requests
{
  "message": "Request added",
  "request": {
    "priority": "High",
    "title": "Internet issue"
  }
}

DELETE /requests/0 (200)
{
  "message": "Request deleted",
  "request": {
    "priority": "High",
    "title": "Internet issue"
  }
}

DELETE /requests/99 (404)
{
  "error": "Request not found"
}
```

Task 4 observation: Deleting a request by valid index removed the item correctly and returned deleted data in the response, confirming expected delete behavior. Attempting to delete with an invalid index produced a clear error response without crashing the server, which demonstrated safe deletion and reliable error handling.

TASK 5:

```python
# Task 5: AI-Assisted API Documentation and Readability
# Implemented through detailed docstrings and inline comments across endpoints.
if __name__ == "__main__":
    # debug=True is useful for lab development; disable in production.
    app.run(debug=True)
```

OUTPUT:

Task 5 - API Documentation

This page documents endpoint paths, HTTP methods, and sample request/response formats.

Back to Live Test Page

Status Endpoint

GET /status

```
{"status": "API is running"}
```

Requests Collection

GET /requests

```
[]
```

POST /requests

Request body:

```
{"title": "Internet issue", "priority": "High"}
```

Response:

```
{"message": "Request added", "request": {"title": "Internet issue", "priority": "High"}}
```

Single Request by Index

PUT /requests/<index>

Example body:

```
{"priority": "Critical"}
```

Success response:

```
{"message": "Request updated", "request": {...}}
```

Error response (404):

```
{"error": "Request not found"}
```

DELETE /requests/<index>

Success response:

```
{"message": "Request deleted", "request": {...}}
```

Search ENG 14:03

Task 5 observation: The API became much easier to understand after adding meaningful docstrings and inline comments for methods, inputs, and responses. A live documentation-style page was also provided to display endpoint paths, HTTP methods, and sample request/response formats, improving readability and making demonstration and reporting more effective.